

A

Project Report On
“College Digital Library Search System”



Department of Computer Science & Engineering

Course Title: Information Retrieval (MC460404)

Branch: MCA (Data Science & Informatics)

Submitted By:

Name	Roll Number
Sourabh Dahiya	2446001
Rishabh Bhardwaj	2446002
Nitin Chourasiya	2446006
Anoop Dwivedi	2446007
Harsh Singhaniya	2446008
Meet Kumar Sarkar	2446009

Submitted To:

Dr. Adla Sanober

TABLE OF CONTENTS

S.No.	Contents	Page No
1.	Abstract	3
2.	Introduction	4
3.	Objective	5
4.	Theory	6
4.1	Information Retrieval System	6
4.2	Document Representation and Preprocessing	6
4.3	Boolean Retrieval Model	6
4.4	TF-IDF (Term Frequency–Inverse Document Frequency)	7
4.5	Ranking Mechanism	8
4.6	Evaluation Metrics	8
4.7	Graphical User Interface (GUI) using Tkinter	8
5.	Algorithm	9-11
6.	Implementation	11-14
7.	Results	15-16
8.	Conclusion	17
9.	Individual Contribution	18
10.	References	18

Abstract

The rapid growth of digital academic resources has created a need for efficient and accurate information retrieval systems. This project presents the design and implementation of a search system for a college digital library that enables users to retrieve books using keyword-based queries. The system integrates Boolean filtering techniques to refine search results based on logical conditions such as AND, OR, and NOT, ensuring precise document selection. Additionally, a Term Frequency–Inverse Document Frequency (TF-IDF) ranking model is implemented to prioritize and display the most relevant documents according to user queries.

The developed system features a user-friendly search interface that allows users to input queries and view filtered and ranked results efficiently. To evaluate the effectiveness of the retrieval system, standard performance metrics such as precision, recall, and F1-score are employed. Experimental results demonstrate the system's ability to improve search accuracy and relevance compared to basic keyword matching approaches.

This project highlights the practical application of core information retrieval techniques and provides a scalable solution for academic digital libraries, enhancing user experience and access to relevant scholarly information.

Introduction

In the modern digital era, the exponential growth of academic content has made efficient information retrieval an essential requirement for educational institutions. College digital libraries act as centralized repositories that store vast amounts of scholarly information. However, as the volume of data increases, users often face challenges in locating relevant documents quickly and accurately. This creates the need for intelligent search systems that can efficiently organize, filter, and rank information based on user queries.

Information Retrieval (IR) systems are specifically designed to address these challenges by enabling users to search and retrieve relevant documents from large datasets. Basic search techniques that rely solely on keyword matching are often insufficient, as they may return irrelevant results or fail to prioritize the most useful documents. To overcome these limitations, more advanced retrieval techniques are required to improve both precision and relevance in search results.

This project presents the design and implementation of a search system for a college digital library that incorporates both Boolean filtering and ranking mechanisms. Boolean filtering allows users to refine their searches using logical operators such as AND, OR, and NOT. These operators help users construct more specific queries, thereby narrowing down the search results and eliminating irrelevant documents. For instance, a user can search for documents containing multiple keywords or exclude certain terms to better match their information needs.

In addition to filtering, the system utilizes the Term Frequency–Inverse Document Frequency (TF-IDF) model to rank documents based on their relevance. TF-IDF is a widely used statistical technique in information retrieval that evaluates the importance of a word in a document relative to a collection of documents. By assigning higher weights to significant and less frequent terms, the system ensures that the most relevant documents are displayed at the top of the search results, improving the overall user experience.

To enhance usability and interaction, the system is implemented with a graphical user interface (GUI) developed using Python's Tkinter library. The GUI provides a simple and intuitive platform where users can input search queries, apply Boolean filters, and view ranked results in an organized manner. This user-friendly interface makes the system accessible even to users with minimal technical knowledge, bridging the gap between complex retrieval algorithms and practical usage.

Furthermore, the project emphasizes evaluating the effectiveness of the search system using standard performance metrics such as precision, recall, and F1-score. These metrics help measure the accuracy and efficiency of the system in retrieving relevant documents while minimizing irrelevant ones. The evaluation process provides insights into the system's performance and highlights areas for potential improvement.

The system demonstrates the practical application of core information retrieval techniques while improving accessibility to academic resources. This project also lays the groundwork for future enhancements, such as incorporating machine learning-based ranking models or semantic search capabilities.

Objective

The primary objective of this project is to design and develop an efficient and user-friendly search system for a college digital library that improves the process of retrieving academic documents. The system aims to integrate fundamental information retrieval techniques with an intuitive graphical interface to enhance both functionality and user experience.

The specific objectives of this project are as follows:

1. To Develop a Keyword-Based Search System

The system is designed to allow users to search for academic documents such as books and research articles using simple keyword queries. This ensures easy access to information without requiring complex search knowledge.

2. To Implement Boolean Filtering

One of the key objectives is to incorporate Boolean logic (AND, OR, NOT) into the search process. This enables users to refine their queries, combine multiple keywords, and exclude irrelevant terms, resulting in more accurate and controlled search results.

3. To Apply TF-IDF Ranking Mechanism

The project aims to implement the Term Frequency–Inverse Document Frequency (TF-IDF) technique to rank retrieved documents based on their relevance to the user's query. This helps prioritize the most meaningful and useful documents at the top of the results list.

4. To Design a User-Friendly GUI Using Tkinter

Another important objective is to develop an interactive and intuitive graphical user interface (GUI) using Python's Tkinter library. The interface should allow users to:

- Enter search queries
- Apply Boolean filters
- View ranked search results clearly

This enhances usability and ensures accessibility for users with varying levels of technical expertise.

5. To Evaluate System Performance

The system is evaluated using standard information retrieval metrics such as:

- Precision
- Recall
- F1-Score

These metrics help measure the effectiveness, accuracy, and reliability of the search system in retrieving relevant documents.

6. To Improve Search Efficiency and Accuracy

The project aims to reduce the retrieval of irrelevant documents while increasing the relevance of search results. This ensures that users can quickly find the information they need with minimal effort.

Theory

The development of the digital library search system is based on core concepts of Information Retrieval (IR). This section explains the theoretical foundations and techniques used in the project, including document representation, Boolean filtering, TF-IDF ranking, and evaluation metrics.

1. Information Retrieval System

An Information Retrieval (IR) system is designed to store, manage, and retrieve relevant information from a large collection of documents based on user queries. Unlike traditional database systems that rely on exact matches, IR systems focus on retrieving documents that are most relevant to the user's needs.

The main components of an IR system include:

- Document collection (corpus)
- Text preprocessing
- Query processing
- Document retrieval and ranking

2. Document Representation and Preprocessing

Before performing search operations, documents must be converted into a structured format. This involves several preprocessing steps:

- **Tokenization:** Breaking text into individual words or terms
- **Stop-word Removal:** Eliminating common words such as “the”, “is”, and “and”
- **Stemming/Lemmatization:** Reducing words to their root form (e.g., “running” → “run”)

These steps help reduce noise in the data and improve the efficiency and accuracy of the retrieval process.

3. Boolean Retrieval Model

The Boolean Retrieval Model allows users to refine search queries using logical operators. It is used in this project to filter documents before ranking.

The main operators include:

- **AND**: Retrieves documents containing all specified terms
- **OR**: Retrieves documents containing at least one of the terms
- **NOT**: Excludes documents containing specific terms

Example:

- Query: “*machine AND learning*” → returns documents containing both terms
- Query: “*data OR mining*” → returns documents containing either term
- Query: “*data NOT mining*” → excludes documents containing “mining”

This model ensures precise filtering but does not provide ranking by itself.

4. TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF is used in the system to rank documents based on their relevance to the query. It assigns weights to terms depending on their importance within a document and across the document collection.

a. Term Frequency (TF)

Measures how often a term appears in a document:

$$TF(t, d) = \frac{\text{Frequency of term } t \text{ in document } d}{\text{Total terms in document}}$$

b. Inverse Document Frequency (IDF)

Measures how unique or important a term is across all documents:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

Where:

- N = Total number of documents
- $df(t)$ = Number of documents containing term t

c. TF-IDF Score

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Documents with higher TF-IDF scores for query terms are considered more relevant and are ranked higher in the search results.

5. Ranking Mechanism

After applying Boolean filtering, the system calculates TF-IDF scores for the remaining documents. The final ranking is based on the cumulative TF-IDF scores of query terms within each document.

This approach ensures:

- Filtered results (using Boolean logic)
- Ranked output (using TF-IDF scores)

6. Evaluation Metrics

To evaluate the effectiveness of the search system, the following standard metrics are used:

a. Precision

Measures how many retrieved documents are relevant:

$$Precision = \frac{\text{Relevant documents retrieved}}{\text{Total documents retrieved}}$$

b. Recall

Measures how many relevant documents are successfully retrieved:

$$Recall = \frac{\text{Relevant documents retrieved}}{\text{Total relevant documents}}$$

c. F1-Score

Provides a balance between precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

7. Graphical User Interface (GUI) using Tkinter

The system includes a graphical user interface developed using Python's Tkinter library. It allows users to interact with the search system easily.

Key features include:

- Input field for entering search queries
- Support for Boolean operators
- Display of filtered and ranked results
- Simple and user-friendly layout

Algorithm

1. Pre Processing:

Input: text (string)

Output: list of tokens

1. Convert text to lowercase
2. Remove all non-alphabetic characters
3. Split text into words (tokens)
4. Return list of tokens

2. Boolean Search:

Input: query (string), documents (list)

Output: filtered_documents (list)

1. Convert query to lowercase
2. Identify operator:
 - if "and" in query → operator = AND
 - else if "or" in query → operator = OR
 - else if "not" in query → operator = NOT
 - else → operator = SINGLE
3. Extract query terms
4. Initialize result list = []
5. For each document in documents:
 - a. Preprocess document → words
 - b. Convert words to set
 - c. If operator = AND:
 - if all query terms in words → add document
 - d. If operator = OR:
 - if any query term in words → add document
 - e. If operator = NOT:
 - if none of the terms in words → add document
 - f. If SINGLE:
 - if any term matches → add document
6. Return result list

3. TF-IDF Ranking:

Input: query (string), documents (list)

Output: ranked_documents (list of (document, score))

1. If documents list is empty:
 return empty list
2. Initialize TF-IDF vectorizer
3. Convert documents into TF-IDF matrix
4. Convert query into TF-IDF vector
5. Compute similarity scores:
 $\text{score} = \text{document_vector} * \text{query_vector}$
6. Pair each document with its score
7. Sort documents in descending order of score
8. Return ranked list

4. Evaluation Matrix:

Input: retrieved (list), relevant (list)

Output: precision, recall, f1

1. Convert retrieved and relevant into sets
2. Compute True Positives (TP):
 $\text{TP} = \text{size of (retrieved} \cap \text{relevant)}$
3. Precision:
 $\text{precision} = \text{TP} / \text{size of retrieved}$
4. Recall:
 $\text{recall} = \text{TP} / \text{size of relevant}$
5. F1-score:
 if $\text{precision} + \text{recall} = 0$:
 $\text{f1} = 0$
 else:
 $\text{f1} = 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$
5. Return precision, recall, f1

6. Overall Search System:

Input: query (string), documents (list)

Output: ranked_results, evaluation_metrics

1. If query is empty:
 return "Enter query"
2. Apply Boolean_Search(query, documents)
 → filtered_docs
3. Apply Rank_Documents(query, filtered_docs)
 → ranked_docs
4. If ranked_docs is empty:
 return "No results found"
5. Define relevant documents (ground truth)
6. Extract retrieved documents from ranked_docs
7. Compute evaluation:
 precision, recall, f1 = Evaluate(retrieved, relevant)
8. Display ranked_docs and evaluation metrics
9. End

Implementation

Main.py:

```
search.py > ...
1 import re
2 from sklearn.feature_extraction.text import TfidfVectorizer
3
4 def preprocess(text):
5     text = text.lower()
6     text = re.sub(r'[^a-z ]', '', text)
7     return text.split()
8
9 def boolean_search(query, docs):
10     query = query.lower()
11
12     # ---- Detect query type ----
13     if query.startswith("not "):
14         # Case: NOT banana
15         exclude_terms = preprocess(query[4:])
16         operator = "not_only"
17
18     elif " and " in query:
19         terms = [t.strip() for t in query.split(" and ")]
20         operator = "and"
21     elif " or " in query:
22         terms = [t.strip() for t in query.split(" or ")]
23         operator = "or"
24     elif " not " in query:
25         # Case: apple NOT banana
26         include_part, exclude_part = query.split(" not ", 1)
27         include_terms = preprocess(include_part)
28         exclude_terms = preprocess(exclude_part)
29         operator = "not"
30     else:
31         terms = preprocess(query)
32         operator = "single"
33
34     results = []
35
36     for doc in docs:
37         processed_doc = preprocess(doc)
38         words = set(processed_doc)
39
40         # Exact match boost
41         if preprocess(query) == processed_doc:
42             results.insert(0, doc)
43             continue
44
45         if operator == "and":
46             if all(term in words for term in terms):
47                 results.append(doc)
48
49         elif operator == "or":
50             if any(term in words for term in terms):
51                 results.append(doc)
52
53         elif operator == "not":
54             # A NOT B → include A, exclude B
55             if (all(term in words for term in include_terms)
56                 and not any(term in words for term in exclude_terms)):
57                 results.append(doc)
58
59         elif operator == "not_only":
60             # NOT B → exclude B
61             if not any(term in words for term in exclude_terms):
62                 results.append(doc)
63
64         else:
65             if any(term in words for term in terms):
66                 results.append(doc)
67
68     return results
69
70
71 def rank_documents(query, docs):
72     if not docs:
73         return []
74
75     vectorizer = TfidfVectorizer()
76     tfidf_matrix = vectorizer.fit_transform(docs)
77     query_vec = vectorizer.transform([query])
78
79     scores = (tfidf_matrix @ query_vec.T).toarray()
80
81     ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)
82
83     return ranked
```

GUI.py:

```
1  import tkinter as tk
2  from tkinter import ttk
3  from data import documents
4  from search import boolean_search, rank_documents
5  from evaluation import evaluate
6
7  def start_app():
8
9      def search():
10         query = entry.get()
11         # Clear old results
12         for widget in result_frame.winfo_children():
13             widget.destroy()
14
15         if not query.strip():
16             show_message("⚠ Enter a query")
17             return
18
19         # Boolean + Ranking
20         filtered = boolean_search(query, documents)
21         ranked = rank_documents(query, filtered)
22
23         if not ranked:
24             show_message("✖ No results found")
25             return
26
27         # Transition Effect (delayed row rendering)
28         for i, (doc, score) in enumerate(ranked):
29             root.after(i * 120, lambda d=doc, s=score:
30                 create_row(d, s))
31
32         # Evaluation after results
33         relevant = ["machine learning basics"]
34
35         precision, recall, f1 = evaluate(
36             [doc for doc, _ in ranked],
37             relevant
38         )
39
40         root.after(len(ranked) * 120, lambda:
41             show_evaluation(precision, recall, f1))
42
43
44     def create_row(doc, score):
45         row = tk.Frame(result_frame, bg="white")
46         row.pack(fill="x", pady=2, padx=10)
47
48         # Hover effect
49         def on_enter(e):
50             row.config(bg="■ #eaf2f8")
51
52         def on_leave(e):
53             row.config(bg="white")
54
55         row.bind("<Enter>", on_enter)
56         row.bind("<Leave>", on_leave)
57
58         # Book name
59         tk.Label(row, text=doc,
60                 font=("Segoe UI", 11),
61                 bg="white", fg="■ #2c3e50",
62                 anchor="w").pack(side="left", padx=10, pady=5)
63
64         # Score
65         tk.Label(row,
66                 text=f"{score[0]:.4f}",
67                 font=("Segoe UI", 10, "bold"),
68                 bg="white", fg="■ #27ae60",
69                 anchor="e").pack(side="right", padx=10)
```

```

72 def show_evaluation(p, r, f1):
73     separator = tk.Frame(result_frame, height=2, bg="#dcdcdc")
74     separator.pack(fill="x", pady=10, padx=10)
75
76     tk.Label(result_frame, text="Evaluation Metrics",
77             font=("Segoe UI", 12, "bold"),
78             bg="#f4f6f7").pack(anchor="w", padx=12)
79
80     tk.Label(result_frame, text=f"Precision: {p:.2f}",
81             bg="#f4f6f7", fg="#8e44ad").pack(anchor="w", padx=12)
82
83     tk.Label(result_frame, text=f"Recall: {r:.2f}",
84             bg="#f4f6f7", fg="#8e44ad").pack(anchor="w", padx=12)
85
86     tk.Label(result_frame, text=f"F1 Score: {f1:.2f}",
87             bg="#f4f6f7", fg="#8e44ad").pack(anchor="w", padx=12, pady=(0,5))
88
89
90 def show_message(msg):
91     tk.Label(result_frame, text=msg,
92             fg="red",
93             bg="#f4f6f7",
94             font=("Segoe UI", 11)).pack(pady=10)
95
96
97 # MAIN WINDOW
98 root = tk.Tk()
99 root.title("College Digital Library")
100 root.geometry("780x560")
101 root.configure(bg="#f4f6f7")
102
103 # TITLE
104 tk.Label(root, text="College Digital Library Search",
105         font=("Segoe UI", 18, "bold"),
106         bg="#f4f6f7", fg="#2c3e50").pack(pady=15)
107
108 # SEARCH BAR
109 top_frame = tk.Frame(root, bg="#f4f6f7")
110 top_frame.pack(pady=10)
111
112 inner = tk.Frame(top_frame, bg="#f4f6f7")
113 inner.pack()
114
115 entry = ttk.Entry(inner, width=40, font=("Segoe UI", 12))
116 entry.grid(row=0, column=0, padx=6)
117
118 tk.Button(inner,
119         text="Search",
120         font=("Segoe UI", 11, "bold"),
121         bg="#1abc9c",
122         fg="white",
123         activebackground="#16a085",
124         relief="flat",
125         padx=18,
126         pady=6,
127         cursor="hand2",
128         command=search).grid(row=0, column=1, padx=6)
129
130 # SCROLLABLE AREA
131 container = tk.Frame(root)
132 container.pack(fill="both", expand=True)
133
134 canvas = tk.Canvas(container, bg="#f4f6f7", highlightthickness=0)
135 scrollbar = tk.Scrollbar(container, command=canvas.yview)
136
137 result_frame = tk.Frame(canvas, bg="#f4f6f7")
138
139 result_frame.bind(
140     "<Configure>",
141     lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
142 )
143
144 canvas.create_window((0, 0), window=result_frame, anchor="nw")
145 canvas.configure(yscrollcommand=scrollbar.set)
146
147 canvas.pack(side="left", fill="both", expand=True)
148 scrollbar.pack(side="right", fill="y")
149
150 root.mainloop()

```

Results

College Digital Library Search

Machine

machine learning basics	0.3952
advanced machine learning techniques	0.3042
artificial intelligence and machine learning	0.2704
pattern recognition and machine learning	0.2704
introduction to machine learning algorithms	0.2564

Evaluation Metrics
Precision: 0.20
Recall: 1.00
F1 Score: 0.33

College Digital Library Search

Data or python

python programming basics	0.3830
advanced python programming	0.3830
deep learning with python	0.3044
data mining techniques	0.2423
big data analytics	0.2262
data structures and algorithms	0.1981
data science for beginners	0.1890
data warehousing and mining concepts	0.1784

Evaluation Metrics
Precision: 0.00
Recall: 0.00
F1 Score: 0.00

College Digital Library

College Digital Library Search

python and programming

Search

python programming basics	0.7093
advanced python programming	0.7093

Evaluation Metrics
Precision: 1.00
Recall: 1.00
F1 Score: 1.00

College Digital Library

College Digital Library Search

not java or python or programming or information

Search

software engineering principles	0.0000
compiler design concepts	0.0000
data structures and algorithms	0.0000
cloud computing fundamentals	0.0000
cyber security basics	0.0000
internet of things concepts	0.0000
blockchain technology introduction	0.0000
distributed systems architecture	0.0000

Evaluation Metrics
Precision: 1.00
Recall: 1.00
F1 Score: 1.00

Conclusion

The developed college digital library search system successfully demonstrates the integration of Boolean retrieval and TF-IDF-based ranking to provide efficient and relevant search results. The system allows users to perform flexible searches using Boolean operators such as AND, OR, and NOT, enabling precise document filtering.

By incorporating TF-IDF vectorization and similarity computation, the system effectively ranks the filtered documents based on their relevance to the user's query. This combination improves the quality of search results compared to using only keyword matching. Additionally, the implementation of evaluation metrics such as Precision, Recall, and F1-score provides a quantitative measure of the system's performance.

Overall, the project highlights the practical application of fundamental Information Retrieval techniques and demonstrates how combining filtering and ranking approaches can enhance search efficiency and accuracy. The system is simple, scalable, and can be further extended to handle larger datasets, advanced ranking models, and real-time search applications.

Individual Contribution

Sourabh Dahiya → TF-IDF

Rishabh Bhardwaj → Evaluation

Nitin Chourasiya → Boolean Search System & Report

Anoop Dwivedi → Rank Documents & Evaluation

Harsh Singhaniya → GUI

Meet Kumar Sarkar → Data Preprocessing and GUI

References

1. Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze. Introduction to Information Retrieval, Cambridge University Press, 2008.